

Universidade do Minho
Escola de Engenharia

Cálculo de Programas

Trabalho Prático (2024/25)

Lic. em Engenharia Informática

Grupo G4

a104445 Alexandre Marques Miranda
a104437 Ana Sá Oliveira
a104179 Sara Azevedo Lopes

janeiro 2025

Preâmbulo

Em Cálculo de Programas pretende-se ensinar a programação de computadores como uma disciplina científica. Para isso parte-se de um repertório de *combinadores* que formam uma álgebra da programação e usam-se esses combinadores para construir programas *composicionalmente*, isto é, agregando programas já existentes.

Na sequência pedagógica dos planos de estudo dos cursos que têm esta disciplina, opta-se pela aplicação deste método à programação em Haskell (sem prejuízo da sua aplicação a outras linguagens funcionais). Assim, o presente trabalho prático coloca os alunos perante problemas concretos que deverão ser implementados em Haskell. Há ainda um outro objectivo: o de ensinar a documentar programas, a validá-los e a produzir textos técnico-científicos de qualidade.

Antes de abordarem os problemas propostos no trabalho, os grupos devem ler com atenção o anexo A onde encontrarão as instruções relativas ao *software* a instalar, etc.

Valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes que utilizem os combinadores de ordem superior estudados na disciplina.

Problema 1

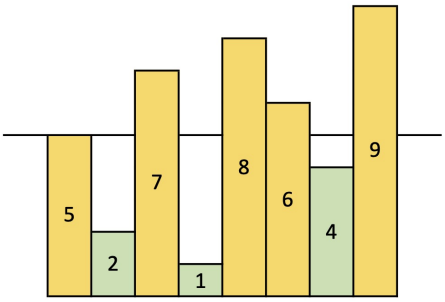
Esta questão aborda um problema que é conhecido pela designação '*H-index of a Histogram*' e que se formula facilmente:

> *O h-index de um histograma é o maior número n de barras do histograma cuja altura é maior ou igual a n.*

Por exemplo, o histograma

$$h = [5, 2, 7, 1, 8, 6, 4, 9]$$

que se mostra na figura



tem $hindex\ h = 5$ pois há 5 colunas maiores que 5. (Não é 6 pois maiores ou iguais que seis só há quatro.)

Pretende-se definida como um catamorfismo, anamorfismo ou hilomorfismo uma função em Haskell

$$hindex :: [Int] \to (Int, [Int])$$

tal que, para $(i, x) = hindex\ h$, i é o H-index de h e x é a lista de colunas de h que para ele contribuem.

A proposta de $hindex$ deverá vir acompanhada de um **diagrama** ilustrativo.

Problema 2

Pelo teorema fundamental da aritmética, todo número inteiro positivo tem uma única factorização prima. For exemplo,

```
primes 455
[5,7,13]
primes 433
[433]
primes 230
[2,5,23]
```

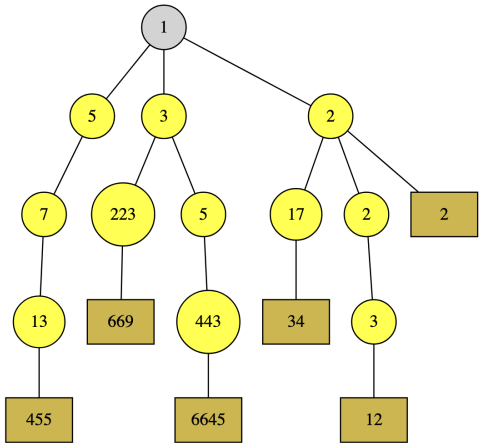
1. Implemente como anamorfismo de listas a função

   $$primes :: \mathbb{Z} \to [\mathbb{Z}]$$

   que deverá, recebendo um número inteiro positivo, devolver a respectiva lista de factores primos.

   A proposta de *primes* deverá vir acompanhada de um **diagrama** ilustrativo.

2. A figura mostra a "*árvore dos primos*" dos números $[455, 669, 6645, 34, 12, 2]$.



   Com base na alínea anterior, implemente uma função em Haskell que faça a geração de uma tal árvore a partir de uma lista de inteiros:

   $$prime_tree :: [\mathbb{Z}] \to Exp\ \mathbb{Z}\ \mathbb{Z}$$

   Sugestão: escreva o mínimo de código possível em *prime_tree* investigando cuidadosamente que funções disponíveis nas bibliotecas que são dadas podem ser reutilizadas.[1]

Problema 3

A convolução $a \star b$ de duas listas a e b — uma operação relevante em computação — está muito bem explicada neste vídeo do canal **3Blue1Brown** do YouTube, a partir de $t = 6:30$. Aí se mostra como, por exemplo:

[1] Pense sempre na sua produtividade quando está a programar — essa atitude será valorizada por qualquer empregador que vier a ter.

$$[1,2,3] \star [4,5,6] = [4,13,28,27,18]$$

A solução abaixo, proposta pelo chatGPT,

$$\begin{aligned}&convolve :: Num\ a \Rightarrow [a] \to [a] \to [a]\\&convolve\ xs\ ys = [sum\ \$\ zipWith\ (*)\ (take\ n\ (drop\ i\ xs))\ ys \mid i \leftarrow [0\,..\,(length\ xs - n)]]\\&\quad \textbf{where}\ n = length\ ys\end{aligned}$$

está manifestamente errada, pois $convolve\ [1,2,3]\ [4,5,6] = [32]$ (!).

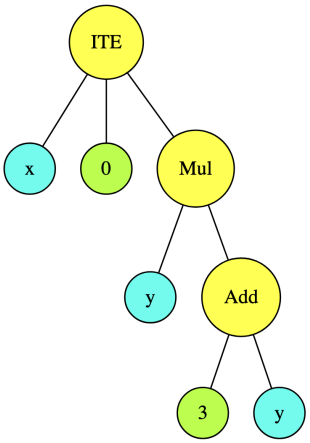
Proponha, explicando-a devidamente, uma solução sua para *convolve*. Valorizar-se-á a economia de código e o recurso aos combinadores *pointfree* estudados na disciplina, em particular a triologia *ana-cata-hilo* de tipos disponíveis nas bibliotecas dadas ou a definir.

Problema 4

Considere-se a seguinte sintaxe (abstrata e simplificada) para **expressões numéricas** (em b) com variáveis (em a),

$$\begin{aligned}&\textbf{data}\ Expr\ b\ a = V\ a \mid N\ b \mid T\ Op\ [Expr\ b\ a]\ \textbf{deriving}\ (Show, Eq)\\&\textbf{data}\ Op = ITE \mid Add \mid Mul \mid Suc\ \textbf{deriving}\ (Show, Eq)\end{aligned}$$

possivelmente condicionais (cf. *ITE*, i.e. o operador condicional "if-then-else"). Por exemplo, a árvore mostrada a seguir



representa a expressão

$$ite\ (V\ \texttt{"x"})\ (N\ 0)\ (multi\ (V\ \texttt{"y"})\ (soma\ (N\ 3)\ (V\ \texttt{"y"}))) \tag{1}$$

— i.e. $\textbf{if}\ x\ \textbf{then}\ 0\ \textbf{else}\ y * (3 + y)$ — assumindo as "helper functions":

$$\begin{aligned}&soma\ x\ y = T\ Add\ [x,y]\\&multi\ x\ y = T\ Mul\ [x,y]\\&ite\ x\ y\ z = T\ ITE\ [x,y,z]\end{aligned}$$

No anexo E propôe-se uma base para o tipo Expr (*baseExpr*) e a correspondente algebra *inExpr* para construção do tipo *Expr*.

1. Complete as restantes definições da biblioteca *Expr* pedidas no anexo F.
2. No mesmo anexo, declare *Expr b* como instância da classe *Monad*. **Sugestão**: relembre os exercícios da ficha 12.

3. Defina como um catamorfismo de *Expr* a sua versão monádia, que deverá ter o tipo:

$$mcataExpr :: Monad\ m \Rightarrow (a + (b + (Op, m\ [c])) \to m\ c) \to Expr\ b\ a \to m\ c$$

4. Para se avaliar uma expressão é preciso que todas as suas variáveis estejam instanciadas. Complete a definição da função

$$let_exp :: (Num\ c) \Rightarrow (a \to Expr\ c\ b) \to Expr\ c\ a \to Expr\ c\ b$$

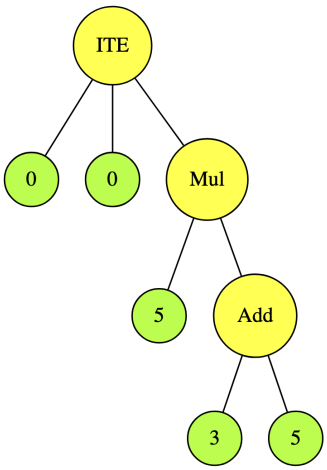
que, dada uma expressão com variáveis em a e uma função que a cada uma dessas variáveis atribui uma expressão ($a \to Expr\ c\ b$), faz a correspondente substituição.[1] Por exemplo, dada

$$f\ \texttt{"x"} = N\ 0$$
$$f\ \texttt{"y"} = N\ 5$$
$$f\ _ = N\ 99$$

ter-se-á

$$let_exp\ f\ e = T\ ITE\ [N\ 1, N\ 0, T\ Mul\ [N\ 5, T\ Add\ [N\ 3, N\ 1]]]$$

isto é, a árvore da figura a seguir:



5. Finalmente, defina a função de avaliação de uma expressão, com tipo

$$evaluate :: (Num\ a, Ord\ a) \Rightarrow Expr\ a\ b \to Maybe\ a$$

que deverá ter em conta as seguintes situações de erro:

(a) *Variáveis* — para ser avaliada, x em *evaluate* x não pode conter variáveis. Assim, por exemplo,

$$evaluate\ e = Nothing$$
$$evaluate\ (let_exp\ f\ e) = Just\ 40$$

para f e e dadas acima.

(b) *Aridades* — todas as ocorrências dos operadores deverão ter o devido número de subexpressões, por exemplo:

$$evaluate\ (T\ Add\ [N\ 2, N\ 3]) = Just\ 5$$
$$evaluate\ (T\ Mul\ [N\ 2]) = Nothing$$

[1] Cf. expressões **let** ... **in**....

Sugestão: de novo se insiste na escrita do mínimo de código possível, tirando partido da riqueza estrutural do tipo *Expr* que é assunto desta questão. Sugere-se também o recurso a diagramas para explicar as soluções propostas.

Anexos

A Natureza do trabalho a realizar

Este trabalho teórico-prático deve ser realizado por grupos de 3 alunos. Os detalhes da avaliação (datas para submissão do relatório e sua defesa oral) são os que forem publicados na página da disciplina na *internet*.

Recomenda-se uma abordagem participativa dos membros do grupo em **todos** os exercícios do trabalho, para assim poderem responder a qualquer questão colocada na *defesa oral* do relatório.

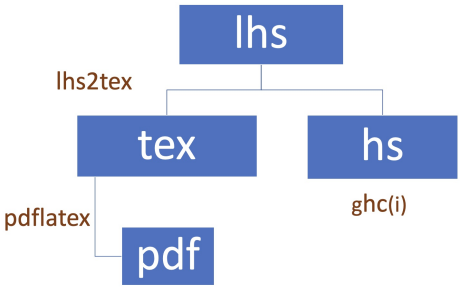
Para cumprir de forma integrada os objectivos do trabalho vamos recorrer a uma técnica de programação dita "literária" [**?**], cujo princípio base é o seguinte:

> *Um programa e a sua documentação devem coincidir.*

Por outras palavras, o **código fonte** e a **documentação** de um programa deverão estar no mesmo ficheiro.

O ficheiro `cp2425t.pdf` que está a ler é já um exemplo de programação literária: foi gerado a partir do texto fonte `cp2425t.lhs`[1] que encontrará no material pedagógico desta disciplina descompactando o ficheiro `cp2425t.zip`.

Como se mostra no esquema abaixo, de um único ficheiro (*lhs*) gera-se um PDF ou faz-se a interpretação do código Haskell que ele inclui:



Vê-se assim que, para além do GHCi, serão necessários os executáveis pdflatex e lhs2TeX. Para facilitar a instalação e evitar problemas de versões e conflitos com sistemas operativos, é recomendado o uso do Docker tal como a seguir se descreve.

B Docker

Recomenda-se o uso do container cuja imagem é gerada pelo Docker a partir do ficheiro `Dockerfile` que se encontra na diretoria que resulta de descompactar `cp2425t.zip`. Este container deverá ser usado na execução do GHCi e dos comandos relativos ao LaTeX. (Ver também a `Makefile` que é disponibilizada.)

[1] O sufixo 'lhs' quer dizer *literate Haskell*.

Após instalar o Docker e descarregar o referido zip com o código fonte do trabalho, basta executar os seguintes comandos:

```
$ docker build -t cp2425t .
$ docker run -v ${PWD}:/cp2425t -it cp2425t
```

NB: O objetivo é que o container seja usado *apenas* para executar o GHCi e os comandos relativos ao LaTeX. Deste modo, é criado um *volume* (cf. a opção `-v ${PWD}:/cp2425t`) que permite que a diretoria em que se encontra na sua máquina local e a diretoria `/cp2425t` no container sejam partilhadas.

Pretende-se então que visualize/edite os ficheiros na sua máquina local e que os compile no container, executando:

```
$ lhs2TeX cp2425t.lhs > cp2425t.tex
$ pdflatex cp2425t
```

lhs2TeX é o pre-processador que faz "pretty printing" de código Haskell em LaTeX e que faz parte já do container. Alternativamente, basta executar

```
$ make
```

para obter o mesmo efeito que acima.

Por outro lado, o mesmo ficheiro `cp2425t.lhs` é executável e contém o "kit" básico, escrito em Haskell, para realizar o trabalho. Basta executar

```
$ ghci cp2425t.lhs
```

Abra o ficheiro `cp2425t.lhs` no seu editor de texto preferido e verifique que assim é: todo o texto que se encontra dentro do ambiente

```
\begin{code}
...
\end{code}
```

é seleccionado pelo GHCi para ser executado.

C Em que consiste o TP

Em que consiste, então, o *relatório* a que se referiu acima? É a edição do texto que está a ser lido, preenchendo o anexo F com as respostas. O relatório deverá conter ainda a identificação dos membros do grupo de trabalho, no local respectivo da folha de rosto.

Para gerar o PDF integral do relatório deve-se ainda correr os comando seguintes, que actualizam a bibliografia (com BibTeX) e o índice remissivo (com `makeindex`),

```
$ bibtex cp2425t.aux
$ makeindex cp2425t.idx
```

e recompilar o texto como acima se indicou. (Como já se disse, pode fazê-lo correndo simplesmente `make` no container.)

No anexo E disponibiliza-se algum código Haskell relativo aos problemas que são colocados. Esse anexo deverá ser consultado e analisado à medida que isso for necessário.

Deve ser feito uso da programação literária para documentar bem o código que se desenvolver, em particular fazendo diagramas explicativos do que foi feito e tal como se explica no anexo D que se segue.

D Como exprimir cálculos e diagramas em LaTeX/lhs2TeX

Como primeiro exemplo, estudar o texto fonte (lhs) do que está a ler[1] onde se obtém o efeito seguinte:[2]

$$
\begin{array}{ll}
 & id = \langle f, g\rangle \\
\equiv & \quad \{\ \text{universal property}\ \} \\
 & \left\{ \begin{array}{l} \pi_1 \cdot id = f \\ \pi_2 \cdot id = g \end{array} \right. \\
\equiv & \quad \{\ \text{identity}\ \} \\
 & \left\{ \begin{array}{l} \pi_1 = f \\ \pi_2 = g \end{array} \right. \\
\Box &
\end{array}
$$

Os diagramas podem ser produzidos recorrendo à *package* xymatrix, por exemplo:

$$
\begin{array}{ccc}
\mathbb{N}_0 & \xleftarrow{\text{in}} & 1 + \mathbb{N}_0 \\
{\scriptstyle (\!|g|\!)}\downarrow & & \downarrow{\scriptstyle id + (\!|g|\!)} \\
B & \xleftarrow[g]{} & 1 + B
\end{array}
$$

E Código fornecido

Problema 1

$$h :: [Int]$$

Problema 4

Definição do tipo:

$$inExpr = [V, [N, \widehat{T}]]$$
$$baseExpr\ g\ h\ f = g + (h + id \times \text{map}\ \ f)$$

Exemplos de expressões:

$$e = ite\ (V\ \texttt{"x"})\ (N\ 0)\ (multi\ (V\ \texttt{"y"})\ (soma\ (N\ 3)\ (V\ \texttt{"y"})))$$
$$i = ite\ (V\ \texttt{"x"})\ (N\ 1)\ (multi\ (V\ \texttt{"y"})\ (soma\ (N\ (3\ /\ 5))\ (V\ \texttt{"y"})))$$

Exemplo de teste:

$$teste = evaluate\ (let_exp\ f\ i) \equiv Just\ (26\ /\ 245)$$
$$\textbf{where}\ f\ \texttt{"x"} = N\ 0; f\ \texttt{"y"} = N\ (1\ /\ 7)$$

[1] Procure e.g. por `"sec:diagramas"`.

[2] Exemplos tirados de [**?**].

F Soluções dos alunos

Os alunos devem colocar neste anexo as suas soluções para os exercícios propostos, de acordo com o "layout" que se fornece. Não podem ser alterados os nomes ou tipos das funções dadas, mas pode ser adicionado texto ao anexo, bem como diagramas e/ou outras funções auxiliares que sejam necessárias.

Importante: Não pode ser alterado o texto deste ficheiro fora deste anexo.

Problema 1

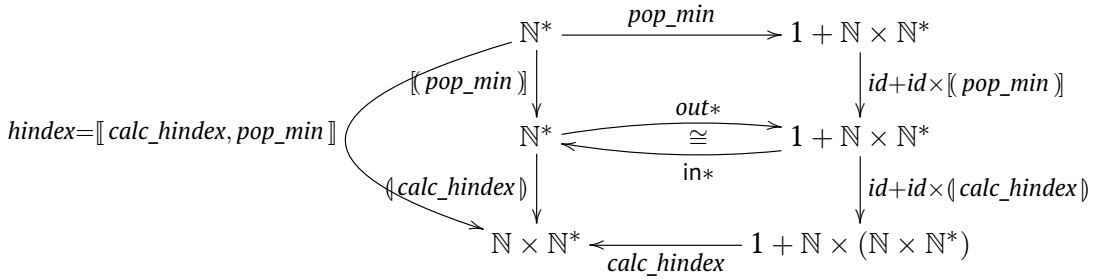
Para resolver o problema do cálculo do H-index de um histograma, foi desenvolvida a seguinte solução em Haskell:

$$
\begin{aligned}
&hindex = hyloList\ calc_hindex\ pop_min\\
&pop_min\ [\,] = i_1\ ()\\
&pop_min\ xs = (i_2 \cdot \langle \pi_1, \widehat{delete} \rangle \cdot \langle minimum, id \rangle)\ xs\\
&calc_hindex = [\underline{0}, [\,], \cdot]\\
&\quad (cond\ (\widehat{(\geqslant)}) \cdot (id \times \mathsf{succ}\ \cdot \pi_1))\\
&\qquad \langle \mathsf{succ}\ \cdot \pi_1 \cdot \pi_2, \mathsf{conc} \cdot swap \cdot (singl \times \pi_2) \rangle\\
&\qquad \pi_2)
\end{aligned}
$$

A função `hindex` aplica um hilomorfismo (`hyloList`) para determinar o H-index de uma lista representando o histograma. O cálculo baseia-se nas seguintes funções:

- `g`: Uma função anamórfica que decompõe o histograma em subestruturas, identificando a barra com a menor altura e preparando os dados para processamento.
- `f`: Uma função catamórfica que processa a estrutura gerada por `g` para calcular o H-index e identificar as barras que contribuem para o resultado.

O uso de *combinadores* como `hyloList`, `split`, e `cond` torna a solução elegante e composicional, refletindo a abordagem funcional desta disciplina.



Exemplo de execução do algoritmo que desenvolvemos

Vamos calcular o **H-index** para a lista `h = [5, 2, 7, 1, 8, 6, 4, 9]`:

- `n`: o menor elemento da lista atual (fornecido por `g`).
- `k`: o índice acumulado até o momento.
- `xs`: a lista restante após remover o menor elemento.

- **Primeira iteração**:

$$g([5,2,7,1,8,6,4,9]) = \texttt{Right (1, [5, 2, 7, 8, 6, 4, 9])}$$
$$f(1,0,[5,2,7,8,6,4,9]) = (1,[5,2,7,8,6,4,9])$$

  Aqui, `n = 1`, `k = 0`, e `xs = [5, 2, 7, 8, 6, 4, 9]`. A condição verificada é:

$$1 \geqslant 0+1 \quad \text{(verdadeiro)}$$

- **Segunda iteração**:

$$g([5,2,7,8,6,4,9]) = \texttt{Right (2, [5, 7, 8, 6, 4, 9])}$$
$$f(2,1,[5,7,8,6,4,9]) = (2,[5,7,8,6,4,9])$$

  Aqui, `n = 2`, `k = 1`, e `xs = [5, 7, 8, 6, 4, 9]`. A condição verificada é:

$$2 \geqslant 1+1 \quad \text{(verdadeiro)}$$

- **Terceira iteração**:

$$g([5,7,8,6,4,9]) = \texttt{Right (4, [5, 7, 8, 6, 9])}$$
$$f(4,2,[5,7,8,6,9]) = (3,[5,7,8,6,9])$$

  Aqui, `n = 4`, `k = 2`, e `xs = [5, 7, 8, 6, 9]`. A condição verificada é:

$$4 \geqslant 2+1 \quad \text{(verdadeiro)}$$

- **Quarta iteração**:

$$g([5,7,8,6,9]) = \texttt{Right (5, [7, 8, 6, 9])}$$
$$f(5,3,[7,8,6,9]) = (4,[7,8,6,9])$$

  Aqui, `n = 5`, `k = 3`, e `xs = [7, 8, 6, 9]`. A condição verificada é:

$$5 \geqslant 3+1 \quad \text{(verdadeiro)}$$

- **Quinta iteração**:

$$g([7,8,6,9]) = \texttt{Right (6, [7, 8, 9])}$$
$$f(6,4,[7,8,9]) = (5,[7,8,9])$$

  Aqui, `n = 6`, `k = 4`, e `xs = [7, 8, 9]`. A condição verificada é:

$$6 \geqslant 4+1 \quad \text{(verdadeiro)}$$

- **Sexta iteração (condição falha)**:

$$g([7,8,9]) = \texttt{Right (7, [8, 9])}$$
$$f(7,5,[8,9]) = (5,[8,9])$$

  Aqui, `n = 7`, `k = 5`, e `xs = [8, 9]`. A condição verificada é:

$$7 \geqslant 5+1 \quad \text{(falso, parar o cálculo)}$$

O cálculo para na sexta iteração, pois a condição avaliada por `f` é falsa. O valor final do **H-index** é, portanto, **5**.

Problema 2

Primeira parte:

$$
\begin{aligned}
&primes = [\!(\, fst_prime \,)\!] \\
&fst_prime\ 1 = i_1\ () \\
&fst_prime\ n = (i_2 \cdot \langle \pi_2, \widehat{\cdot \div \cdot} \rangle \\
&\quad \cdot \langle id, head \cdot \widehat{filter} \\
&\quad \cdot \langle ((\equiv 0) \cdot) \cdot mod, \mathsf{conc} \\
&\quad \cdot \langle (2:) \cdot enumFromThenTo\ 3\ 5 \cdot floor \cdot sqrt \cdot fromIntegral, singl \rangle \rangle \rangle)\ n
\end{aligned}
$$

$$
\begin{array}{ccc}
\mathbb{N} & \xrightarrow{fst_prime} & 1 + \mathbb{N} \times \mathbb{N} \\
{\scriptstyle primes = [\!(\, fst_prime \,)\!]}\downarrow & & \downarrow{\scriptstyle id + id \times [\!(\, fst_prime \,)\!]} \\
\mathbb{N}^* & \xleftarrow[\mathsf{in}*]{} & 1 + \mathbb{N} \times \mathbb{N}^*
\end{array}
$$

Segunda parte:

$$prime_tree = Term\ 1 \cdot untar \cdot \mathsf{map}\ \ \langle primes, id \rangle$$

Problema 3

$$
\begin{aligned}
&convolve :: Num\ a \Rightarrow [a] \to [a] \to [a] \\
&convolve = \bot
\end{aligned}
$$

Problema 4

Definição do tipo:

$$
\begin{aligned}
&outExpr\ (V\ a) = i_1\ a \\
&outExpr\ (N\ b) = (i_2 \cdot i_1)\ b \\
&outExpr\ (T\ op\ l) = (i_2 \cdot i_2)\ (op, l) \\
&recExpr = baseExpr\ id\ id
\end{aligned}
$$

Ana + *cata* + *hylo*:

$$
\begin{aligned}
&cataExpr\ g = g \cdot (recExpr\ (cataExpr\ g)) \cdot outExpr \\
&anaExpr\ g = inExpr \cdot (recExpr\ (anaExpr\ g)) \cdot g \\
&hyloExpr\ h\ g = cataExpr\ h \cdot anaExpr\ g
\end{aligned}
$$

Monad:

$$
\begin{aligned}
&\textbf{instance}\ Functor\ (Expr\ b)\ \textbf{where} \\
&\quad \mathsf{fmap}\ f = cataExpr\ (inExpr \cdot baseExpr\ f\ id\ id) \\
&\textbf{instance}\ Applicative\ (Expr\ b)\ \textbf{where} \\
&\quad (<*>) = aap \\
&\quad pure = return \\
&\textbf{instance}\ Monad\ (Expr\ b)\ \textbf{where} \\
&\qquad \text{-- x >>= g = (u . fmap f) x}
\end{aligned}
$$

```
return = V
  -- e »= g = (cataExpr () . fmap g) e
(V a) >>= g = g a
(N b) >>= g = N b
(T op l) >>= g = T op (map (>>=g) l)
```

Maps: *Monad*: *Let expressions*:

```
let_exp f = (>>=f)
```

Catamorfismo monádico:

```
  -- mcataExpr = undefined
mcataExpr g = g .! (d · recExpr (mcataExpr g) · outExpr)
d :: Monad m ⇒ a + (b + (Op, [m c])) → m (a + (b + (Op, m [c])))
d = [return · i1, [return · i2 · i1, fmap (i2 · i2) · l]]
l :: Monad m ⇒ (Op, [m c]) → m (Op, m [c])
l (op, t) = do
    -- Transformar 't :: [m c]' em 'm [c]' usando 'sequence'
  mcs ← sequence t   -- mcs :: m [c]
  return (op, return mcs)
```

Avaliação de expressões:

```
evaluate = mcataExpr (inMaybe · fu)
fu :: (Num a, Ord a) ⇒ b + (a + (Op, Maybe [a])) → () + a
fu = [i1 (), [i2, evalOp]]
evalOp :: (Num a, Ord a) ⇒ Op → Maybe [a] → () + a
evalOp Suc (Just [x]) = i2 (x + 1)
evalOp Add (Just [x, y]) = i2 (x + y)
evalOp Mul (Just [x, y]) = i2 (x * y)
evalOp ITE (Just [x, y, z]) = if x ≠ 0 then i2 (y) else i2 (z)
evalOp _ _ = i1 ()   -- Qualquer outro caso falha
```